Group Sprint 1

November 3, 2026

[ ]:

[6]:
```
"""Prepare MI training data without fitting any preprocessing on held-out data.

Usage: python prepare_random_forest_data.py
Requires pandas, scikit-learn. Run from directory containing training_data.csv.
"""

import numpy as np
from pathlib import Path
import os
import pandas as pd
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import GridSearchCV, StratifiedKFold
from sklearn.metrics import (
    average_precision_score,
    roc_auc_score,
    recall_score,
    accuracy_score,
    confusion_matrix
)


#ROOT = Path.cwd()
train = pd.read_csv("training_data.csv")
TARGET = 'mortality'
# Later-day therapy counts: exclude for an admission-time prediction.
LATER_DAY = ['R_AB_1_n','R_AB_2_n','R_AB_3_n',
             'NA_R_1_n','NA_R_2_n','NA_R_3_n',
             'NOT_NA_1_n','NOT_NA_2_n','NOT_NA_3_n']
# Exclude these only after confirming they were measured after the intended␣
 ↪prediction time.
# Also audit other treatment and complication variables against the data␣
 ↪dictionary.
```

```
constant = train.drop(columns=[TARGET]).columns[train.drop(columns=[TARGET]).
 ↪nunique(dropna=True) <= 1].tolist()
excluded = sorted(set(constant + [c for c in LATER_DAY if c in train.columns]))
clean = train.drop(columns=excluded)
clean.to_csv(ROOT / 'training_model_ready_unimputed.csv', index=False)
print('Dropped:', excluded)
print('Cleaned shape:', clean.shape)
print('Missing entries remaining:', int(clean.isna().sum().sum()))

# Example: fit imputation only on training X; NEVER fit on validation/test.
X_train = clean.drop(columns=[TARGET])
y_train = clean[TARGET]
# Numeric-coded clinical features: median is a practical starting baseline.
# For categorical-coded columns, consider most-frequent or explicit missing␣
 ↪category,
# after checking the original data dictionary.

print("Predictor columns:", X_train.shape[1])
print("Missing target values:", y_train.isnull().sum())

# Later: model.fit(X_train, y_train)
# Then model.predict_proba(X_validation)[:, 1] with identical feature columns
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[6], line 36
     32 # Also audit other treatment and complication variables against the dat␣
 ↪dictionary.
     33 constant = train.drop(columns=[TARGET]).columns[train.
 ↪drop(columns=[TARGET]).nunique(dropna=True) <= 1].tolist()
     34 excluded = sorted(set(constant + [c for c in LATER_DAY if c in train.
 ↪columns]))
     35 clean = train.drop(columns=excluded)
---> 36 clean.to_csv(ROOT / 'training_model_ready_unimputed.csv', index=False)
     37 print('Dropped:', excluded)
     38 print('Cleaned shape:', clean.shape)
     39 print('Missing entries remaining:', int(clean.isna().sum().sum()))

NameError: name 'ROOT' is not defined
```

[9]:
```
from pathlib import Path
import os
import pandas as pd
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.ensemble import RandomForestClassifier
```

```
train = pd.read_csv("training_data.csv")
TARGET = 'mortality'
# Later-day therapy counts: exclude for an admission-time prediction.
LATER_DAY = ['R_AB_1_n','R_AB_2_n','R_AB_3_n',
             'NA_R_1_n','NA_R_2_n','NA_R_3_n',
             'NOT_NA_1_n','NOT_NA_2_n','NOT_NA_3_n']#can change these if we include all the times

# Also audit other treatment and complication variables against the data dictionary. such as constants.
constant = train.drop(columns=[TARGET]).columns[train.drop(columns=[TARGET]).nunique(dropna=True) <= 1].tolist()
excluded = sorted(set(constant + [c for c in LATER_DAY if c in train.columns]))
clean = train.drop(columns=excluded)
clean = train.drop(columns=[])
clean.to_csv(ROOT / 'training_model_ready_unimputed.csv', index=False)
print('Dropped:', excluded)
print('Cleaned shape:', clean.shape)
print('Missing entries remaining:', int(clean.isna().sum().sum()))
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[9], line 20
     16 constant = train.drop(columns=[TARGET]).columns[train.drop(columns=[TARGET]).nunique(dropna=True) <= 1].tolist()
     17 excluded = sorted(set(constant + [c for c in LATER_DAY if c in train.columns]))
     18 clean = train.drop(columns=excluded)
     19 clean = train.drop(columns=[])
---> 20 clean.to_csv(train / 'training_model_ready_unimputed.csv', index=False)
     21 print('Dropped:', excluded)
     22 print('Cleaned shape:', clean.shape)
     23 print('Missing entries remaining:', int(clean.isna().sum().sum()))

File /opt/conda/lib/python3.13/site-packages/pandas/core/ops/common.py:85, in _unpack_zerodim_and_defer.<locals>.new_method(self, other)
     82     other = sanitize_array(other, None)
     83     other = ensure_wrapped_if_datetimelike(other)
---> 85 return method(self, other)

File /opt/conda/lib/python3.13/site-packages/pandas/core/arraylike.py:214, in OpsMixin.__truediv__(self, other)
    212 @unpack_zerodim_and_defer("__truediv__")
    213 def __truediv__(self, other):
--> 214     return self._arith_method(other, operator.truediv)
```

```
File /opt/conda/lib/python3.13/site-packages/pandas/core/frame.py:9151, in␣
 ↪DataFrame._arith_method(self, other, op)
   9147
   9148         self, other = self._align_for_op(other, axis, flex=True,␣
 ↪level=None)
   9149
   9150         with np.errstate(all="ignore"):
-> 9151             new_data = self._dispatch_frame_op(other, op, axis=axis)
   9152         return self._construct_result(new_data, other=other)

File /opt/conda/lib/python3.13/site-packages/pandas/core/frame.py:9183, in␣
 ↪DataFrame._dispatch_frame_op(self, right, func, axis)
   9179
   9180         right = lib.item_from_zerodim(right)
   9181         if not is_list_like(right):
   9182             # i.e. scalar, faster than checking np.ndim(right) == 0
-> 9183             bm = self._mgr.apply(array_op, right=right)
   9184             return self._constructor_from_mgr(bm, axes=bm.axes)
   9185
   9186         elif isinstance(right, DataFrame):

File /opt/conda/lib/python3.13/site-packages/pandas/core/internals/managers.py:
 ↪443, in BaseBlockManager.apply(self, f, align_keys, **kwargs)
    440             kwargs[k] = obj[b.mgr_locs.indexer]
    442 if callable(f):
--> 443     applied = b.apply(f, **kwargs)
    444 else:
    445     applied = getattr(b, f)(**kwargs)

File /opt/conda/lib/python3.13/site-packages/pandas/core/internals/blocks.py:
 ↪347, in Block.apply(self, func, **kwargs)
    341 @final
    342 def apply(self, func, **kwargs) -> list[Block]:
    343     """
    344     apply the function to my values; return a block if we are not
    345     one
    346     """
--> 347     result = func(self.values, **kwargs)
    349     result = maybe_coerce_values(result)
    350     return self._split_op_result(result)

File /opt/conda/lib/python3.13/site-packages/pandas/core/ops/array_ops.py:289,␣
 ↪in arithmetic_op(left, right, op)
    285     _bool_arith_check(op, left, right)  # type: ignore[arg-type]
    287     # error: Argument 1 to "_na_arithmetic_op" has incompatible type
    288     # "Union[ExtensionArray, ndarray[Any, Any]]"; expected "ndarray[Any␣
 ↪Any]"
```

```
--> 289     res_values = _na_arithmetic_op(left, right, op)  # type:␣
 ↪ignore[arg-type]
    291 return res_values

File /opt/conda/lib/python3.13/site-packages/pandas/core/ops/array_ops.py:220,␣
 ↪in _na_arithmetic_op(left, right, op, is_cmp)
    217     func = partial(expressions.evaluate, op)
    219 try:
--> 220     result = func(left, right)
    221 except TypeError:
    222     if not is_cmp and (
    223         left.dtype == object or getattr(right, "dtype", None) == object
    224     ):
   (…)    227         # Don't do this for comparisons, as that will handle␣
 ↪complex numbers
    228         #  incorrectly, see GH#32047

TypeError: ufunc 'divide' not supported for the input types, and the inputs␣
 ↪could not be safely coerced to any supported types according to the casting␣
 ↪rule ''safe''
```

[ ]:
```
X_train = clean.drop(columns=[TARGET])#variables without the mortality we are␣
 ↪predicting
y_train = clean[TARGET]#what we're predicting
print("Predictor columns:", X_train.shape[1])
print("Missing target values:", y_train.isnull().sum())
```

[10]:
```
#FROM HERE
train = pd.read_csv("training_data.csv")
TARGET = 'mortality'
X_train = clean.drop(columns=[TARGET])#variables without the mortality we are␣
 ↪predicting
y_train = clean[TARGET]#what we're predicting
print("Predictor columns:", X_train.shape[1])
print("Missing target values:", y_train.isnull().sum())
```

```
Predictor columns: 107
Missing target values: 0
```

[11]:
```
numerical_cols =␣
 ↪["AGE","S_AD_ORIT","D_AD_ORIT","K_BLOOD","NA_BLOOD","ALT_BLOOD","AST_BLOOD","L_BLOOD","ROE"]
categorical_cols = [col for col in X_train.columns
        if col not in numerical_cols]
#numerical and caterogrical columns for imputation

print("Numerical variables:", len(numerical_cols))
print("Categorical variables:", len(categorical_cols))
```

```
Numerical variables: 9
Categorical variables: 98
```

[12]:
```
#some variables are missing as much as 40% data. Imputing this could sway data␣
 ↪a lot.
# Create imputers
num_imputer = SimpleImputer(strategy="median",add_indicator=False)
cat_imputer = SimpleImputer(strategy="most_frequent",add_indicator=False)

# Make a copy of the training data
X_train_imputed = X_train.copy()

# Impute numerical columns
X_train_imputed[numerical_cols] = num_imputer.
 ↪fit_transform(X_train[numerical_cols])

# Impute categorical columns
X_train_imputed[categorical_cols] = cat_imputer.
 ↪fit_transform(X_train[categorical_cols])
```

[13]:
```
#check imputation worked:
print("Missing values before:",
      X_train.isnull().sum().sum())

print("Missing values after:",
      X_train_imputed.isnull().sum().sum())
```

```
Missing values before: 6387
Missing values after: 0
```

[27]:
```
#random forest check best parameters:

#we have a set seed to make our results repeatable
fit_rf = RandomForestClassifier(n_estimators=300,bootstrap=True,random_state=1)
np.random.seed(1)

param_grid = {'max_depth': [3, 5, 10, None],#here we describe the different␣
 ↪options for our paramters
              'min_samples_leaf': [1, 5, 10],
              'max_features': ['sqrt','log2', 0.5, None],
              'class_weight': [None, 'balanced_subsample']}#since mortality is␣
 ↪only 16% of data


cv = StratifiedKFold(n_splits=5,shuffle=True,random_state=1)#5 means around 30␣
 ↪deaths per fold not 16
```

```
cv_rf = GridSearchCV(estimator=fit_rf, cv=cv,#this tells the algorithm to use␣
 ↪cross validation to validate our
                     param_grid=param_grid,#results and find the best␣
 ↪parameters
                     scoring={'average_precision': 'average_precision',
                              'roc_auc': 'roc_auc',
                              'recall': 'recall',
                              'accuracy': 'accuracy'},
                     refit='average_precision',
                     n_jobs=3)

cv_rf.fit(X_train_imputed, y_train)#performs the fits and remembers best␣
 ↪parameters.
print('Best Parameters using grid search: \n',
      cv_rf.best_params_)
print("\nBest cross-validation average precision:")
print(cv_rf.best_score_)
```

```
Best Parameters using grid search:
 {'class_weight': 'balanced_subsample', 'max_depth': None, 'max_features':
'sqrt', 'min_samples_leaf': 5}

Best cross-validation average precision:
0.6940897515504775
```

[ ]: